

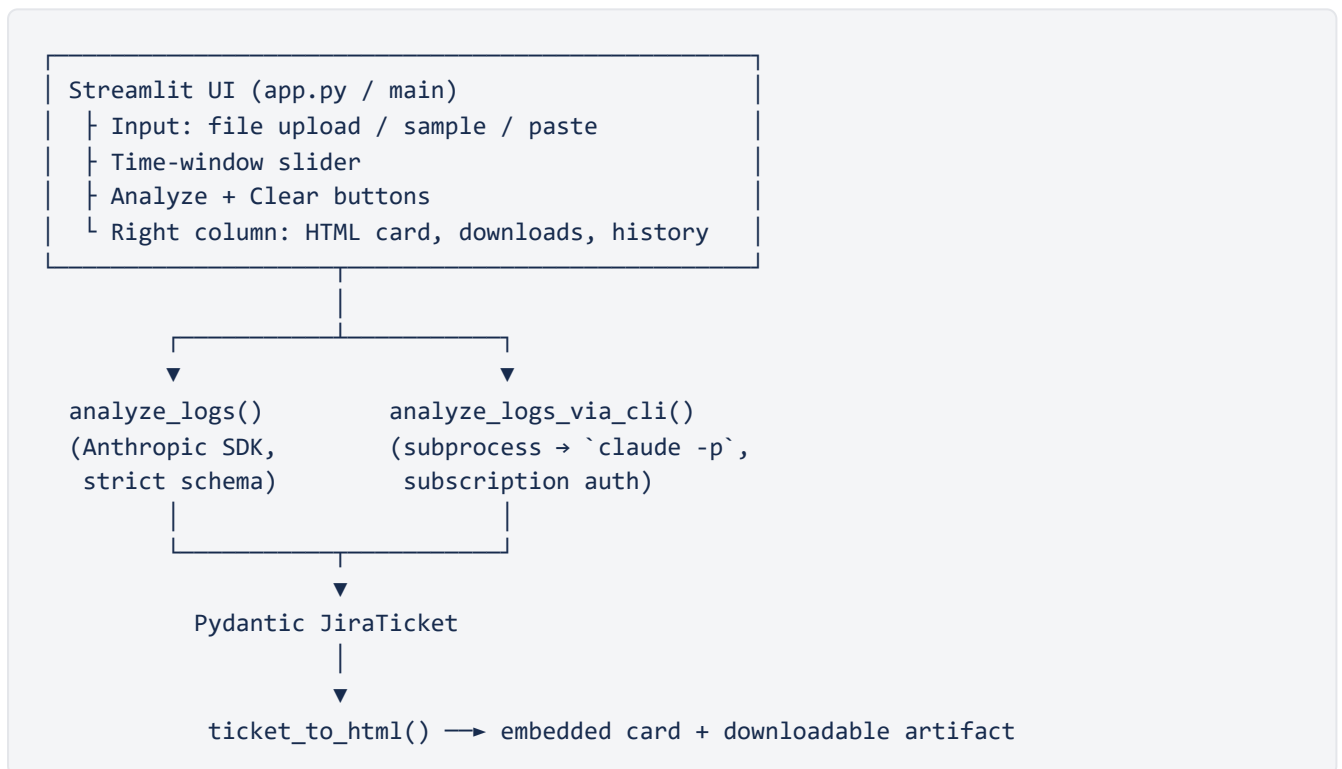
Logs → Jira — Specification

Overview

A Streamlit app that turns raw application logs into a structured, polished Jira-style ticket. Paste, upload, or pick a sample log; optionally narrow to a specific time window; click Analyze; get back a card with title, severity (P0–P3), component, description, repro steps, suggested assignee, labels, and a deduplication-ready error signature.

The app is designed for SRE / on-call workflows where the first 90% of writing a ticket — parsing the stack trace, judging severity, reconstructing the request flow — is repetitive and well-suited to an LLM. The human keeps the final 10% (review and dispatch).

Architecture



Two analysis backends:

1. **API path** (`analyze_logs`) — `client.messages.parse()` with `output_format=JiraTicket`. Strict Pydantic-validated output, fastest (~3–5s), requires `ANTHROPIC_API_KEY` with credits.
2. **CLI fallback** (`analyze_logs_via_cli`) — pipes the prompt via stdin to `claude -p`, scrubs `ANTHROPIC_API_KEY` from the subprocess env so it uses `claude.ai` subscription auth. Slower (~15s), no strict schema validation; output is parsed with a JSON-fence-tolerant extractor.

Currently the UI always uses the CLI path (zero API credits available). The API path is retained for when credits are added.

Data model: `JiraTicket`

Field	Type	Notes
<code>title</code>	<code>str</code>	Action-oriented, under 80 chars
<code>severity</code>	<code>Literal["P0", "P1", "P2", "P3"]</code>	Severity guide in <code>SYSTEM_PROMPT</code>
<code>component</code>	<code>str</code>	Inferred from package paths / service names
<code>description</code>	<code>str</code>	Markdown; rendered as paragraphs in HTML
<code>repro_steps</code>	<code>list[str]</code>	Ordered, reconstructed from request IDs and timestamps
<code>suggested_assignee</code>	<code>str None</code>	Inferred from CODEOWNERS-style package paths
<code>labels</code>	<code>list[str]</code>	3–6 lowercase-kebab tags
<code>error_signature</code>	<code>str</code>	Most diagnostic single line — used for dedup

User flow

1. **Load logs** — upload a `.log` / `.txt` / `.out` / `.json` / `.err`, pick a sample from `sample_logs/`, or paste directly into the textarea.
2. **Optional: narrow time window** — if the input contains ≥ 2 distinct parseable timestamps, a slider appears. Drag the handles to focus on a specific incident window; stack-trace continuation lines ride along with their parent timestamp.
3. **Click Analyze** — spinner ~15s, then the ticket renders in the right column as an embedded HTML card.
4. **Download or revisit** — "Download HTML" / "Download JSON" buttons sit below the card. The last 5 analyses appear under "Recent" and can be re-rendered with one click.

Setup & running

```
# Python 3.11+
pip install -r requirements.txt

# Optional: API path (requires credits)
cp .env.example .env
# edit .env, add ANTHROPIC_API_KEY=sk-ant-...

# Requires the `claude` CLI on PATH for the fallback path
streamlit run app.py
```

Visit <http://localhost:8501>.

Configuration

Knob	Where	Default
<code>MODEL</code>	<code>app.py</code> constant	<code>claude-opus-4-7</code>
<code>SYSTEM_PROMPT</code>	<code>app.py</code> constant	SRE-focused, includes severity guide
<code>ANTHROPIC_API_KEY</code>	<code>.env</code>	unset → CLI fallback
Sample logs	<code>sample_logs/</code>	<code>oom_payment_service.log</code> , <code>db_timeout_search.log</code>
Severity palette	<code>SEVERITY_PALETTE</code> dict	Jira-style red/orange/blue/gray

Known limitations

- **CLI fallback is slow** — ~15s vs ~3–5s on the API path. Acceptable for one-off use, painful for iteration.
- **CLI output is not schema-validated** — if Claude returns malformed JSON, the app falls back to a canned `DEMO_TICKET` so the UI never breaks. Hasn't been observed in practice but possible.
- **Timestamp parser handles ISO-like formats only** — `YYYY-MM-DD[T ]HH:MM:SS[.,;µs]` `[Z|+HH:MM]` , with optional year. syslog (`May 19 14:32:11`), nginx access logs, and other custom formats won't be detected and the slider won't appear.
- **History is in-memory only** — closing the tab or reloading wipes it.
- `ticket.html` **churns** — currently committed to the repo; rewritten on every smoke test or app render. Consider `.gitignore` -ing it.
- **No multi-ticket support** — a log dump with two unrelated incidents is currently collapsed into one ticket.

Future work

In rough order of demo impact:

1. **Push to ticketing system** — Jira, Linear, or GitHub Issues. The single highest-impact change; turns a viewer into a tool.
2. **Multi-ticket grouping** — split distinct root causes into separate tickets and stack them in the right column. Matches the master-prompt spec.
3. **Streaming UI** — fill ticket fields as the model produces them, replacing the blank 15s wait.
4. **Broader timestamp format support** — syslog, nginx, journalctl, plus auto-detection of the dominant format in a paste.
5. **Tests** — pytest for `_parse_ts`, `_filter_logs_by_window`, `_extract_json`, and the HTML escaping in `ticket_to_html`.
6. **Persistence** — SQLite (or just JSON on disk) for history, with dedup by `error_signature` so we don't re-file the same incident twice.
7. **Inline editing** — let the user nudge severity/labels before download or push.